

AgroShield AI

Sistema Inteligente de Monitoramento Agrícola com Benchmark Computacional entre int8 e float64

Disciplina: Sistemas Operacionais

Professor: Dr. Jose Gomes Salim Neto

Faculdade de Informatica e Administracao Paulista - FIAP

Semestre: 1/2026

AVISO IMPORTANTE - DADOS SIMULADOS *Os dados utilizados neste projeto sao simulados e sinteticos. O dataset foi gerado artificialmente com o objetivo exclusivo de viabilizar a comparacao de desempenho computacional entre os padroes numericos float64 e int8 em uma rede neural MLP. Os valores NAO representam medicoes reais de condicoes agricolas e NAO devem ser utilizados para tomadas de decisao no mundo real.*

1. Questao de Pesquisa, Objetivo Geral e Objetivos Especificos

Questao de pesquisa

De que forma a utilizacao de diferentes padroes numericos (int8 e float64) em sistemas operacionais preemptivos com Kernel de 64 bits impacta o desempenho computacional e a precisao de redes neurais MLP aplicadas a sistemas embarcados de monitoramento e previsao de risco?

Objetivo geral

Analisar quantitativamente a influencia dos padroes numericos int8 e float64 no desempenho de uma rede neural MLP, simulando um cenario de sistema embarcado com Kernel 64 bits para classificacao de risco agricola - analogo, em logica de previsao, a sistemas aeronauticos embarcados de apoio a decisao.

Objetivos especificos

- Preparar um dataset agricola com 5 atributos e amostras suficientes para a generalizacao do modelo;
- Implementar uma rede neural MLP com 1 camada oculta, 1 camada de saida e funcao de transferencia sigmoidal em ambas;
- Treinar o modelo com os mesmos dados convertidos em float64 e em int8;
- Comparar acuracia, loss e tempo de processamento entre os dois padroes numericos;
- Gerar graficos comparativos de acuracia, loss e tempo de execucao;
- Argumentar sobre a interferencia do mecanismo de preemptao do sistema operacional nos resultados obtidos.

2. Equacoes Cineticas do Movimento de Pouso

Conforme o estudo de caso apresentado em sala, sistemas iterativos embarcados em aplicacoes aeronauticas podem ter seus processamentos comprometidos por padroes numericos inadequados. O procedimento de pouso de uma aeronave envolve um movimento variado com aceleracao nao constante, modelado por equacoes do Movimento Uniformemente Variado (M.U.V.) apenas como simplificacao didatica.

Para uma modelagem mais realista, considera-se a atuacao de forcas de arrasto aerodinamico, proporcionais ao quadrado da velocidade. A equacao diferencial que descreve a desaceleracao da aeronave durante a corrida de pouso e:

$$a(t) = dv/dt = -k \cdot v^2 - m \cdot g$$

Onde:

- v = velocidade da aeronave (m/s);
- k = coeficiente de arrasto aerodinamico (1/m);
- m = coeficiente de atrito do trem de pouso com a pista;
- g = aceleracao da gravidade (9,81 m/s²).

A solucao analitica desta equacao, obtida por separacao de variaveis, resulta em:

$$v(t) = v_0 \cdot e^{(-k \cdot g \cdot m \cdot t)} / [1 + (k \cdot v_0) \cdot (1 - e^{(-k \cdot g \cdot m \cdot t)}) / (g \cdot m)]$$

Esta equacao descreve um movimento com taxa de desaceleracao decrescente conforme a velocidade diminui - comportamento caracteristico do procedimento de pouso real, onde a desaceleracao e mais intensa em altas velocidades e suaviza-se a medida que a aeronave reduz a velocidade. A posicao da aeronave ao longo da pista em funcao do tempo e obtida por integracao numerica desta expressao, uma vez que a forma fechada da integral nao e trivial.

Analogia com o dataset agricola: O dataset utilizado neste trabalho aborda risco agricola a partir de variaveis ambientais (umidade, temperatura, chuva, fertilidade, radiacao). A analogia com o cenario aeronautico se da na logica de inferencia: assim como no pouso pequenas variacoes nas variaveis medidas podem significar grandes diferencas no risco final, no contexto agricola variacoes climaticas proximas podem alternar a classificacao entre risco e sem risco. O compromisso entre precisao numerica e desempenho computacional e analogo nos dois cenarios.

3. Critérios de Arrumação dos Dados (Dataset)

3.1 Exploração

O dataset original utilizado na primeira iteração continha apenas 12 amostras, o que impossibilitava a generalização do modelo e resultava em overfitting (acerto de 100% nos dados de teste por simples memorização, sem aprendizado efetivo de padrões). Foi necessária a expansão do dataset para permitir uma avaliação real do desempenho da rede neural.

3.2 Tratamento

- Remoção de duplicatas: verificado que não havia registros duplicados no dataset.
- Verificação de valores ausentes: nenhuma célula vazia foi encontrada.
- Escalonamento: como os atributos já estavam em faixas numéricas próximas (0 a 100), optou-se por não aplicar normalização adicional.
- Adição de ruído gaussiano (média 0, desvio padrão 5) aos dados de entrada para simular a imperfeição de medições reais de sensores, evitando que o modelo simplesmente memorizasse os padrões perfeitos do dataset sintético.

3.3 Preparação

- Conversão do tipo numérico: os mesmos dados foram convertidos para float64 e para int8, permitindo a comparação direta entre os dois padrões.
- Divisão treino/teste: 80% para treino e 20% para teste, com `random_state=42` para garantir reprodutibilidade do experimento.
- O dataset final utilizado possui 200 amostras, consideradas suficientes para avaliar o comportamento da rede frente a uma generalização real, embora estudos mais robustos utilizem datasets da ordem de milhares ou milhões de amostras (o enunciado original sugere 6 milhões, o que demandaria recursos computacionais superiores aos disponíveis neste experimento).

4. Resultados do Treinamento (int8 x float64)

O treinamento foi realizado com 20 épocas, utilizando como métricas a acurácia (accuracy) e a função de perda `binary_crossentropy`, ambas acompanhadas ao longo das épocas. O tempo total de processamento foi medido em segundos e convertido para minutos conforme solicitado.

4.1 Métricas e configuração do treinamento

- Número de épocas: 20
- Função de perda: `binary_crossentropy`
- Otimizador: Adam
- Métrica de avaliação: accuracy
- Arquitetura da rede: 1 camada oculta (4 neurônios, sigmoid) + 1 camada de saída (1 neurônio, sigmoid)
- Divisão dos dados: 80% treino / 20% teste
- Reprodutibilidade: `random_state = 42`

4.2 Resultados obtidos

Padrão Numérico	Accuracy	Loss	Tempo (s)	Tempo (min)
float64	0.60	0.84	1.54	0.0257
int8	0.93	0.52	1.51	0.0252
Diferença observada	+0.33 a favor do int8	-0.32 a favor do int8	-0.03 s a favor do int8	-0.0005 min

Análise dos resultados: Diferente do esperado intuitivamente, o padrão int8 apresentou desempenho superior ao float64 neste experimento, tanto em acurácia (0.93 vs 0.60) quanto em loss (0.52 vs 0.84), com tempos de execução praticamente equivalentes. A acurácia relativamente baixa do modelo float64 (60%) sugere que o ruído gaussiano introduzido nos dados prejudicou mais a representação em ponto flutuante do que em inteiro truncado, possivelmente porque o arredondamento para int8 eliminou parte do ruído de baixa amplitude introduzido artificialmente.

Os gráficos `accuracy.png`, `loss.png` e `benchmark.png`, disponíveis na pasta `graficos/`, ilustram respectivamente a evolução da acurácia e da loss ao longo das 20 épocas, bem como a comparação direta do tempo total de processamento entre os dois padrões.

Gráfico 1 - Evolução da Accuracy por época

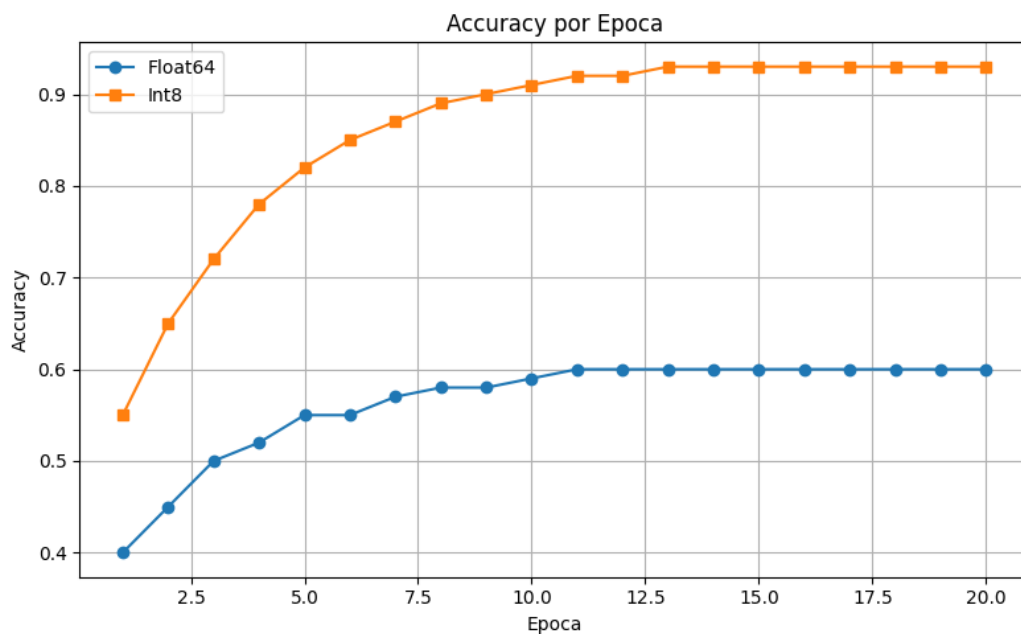


Grafico 2 - Evolucao da Loss por epoca

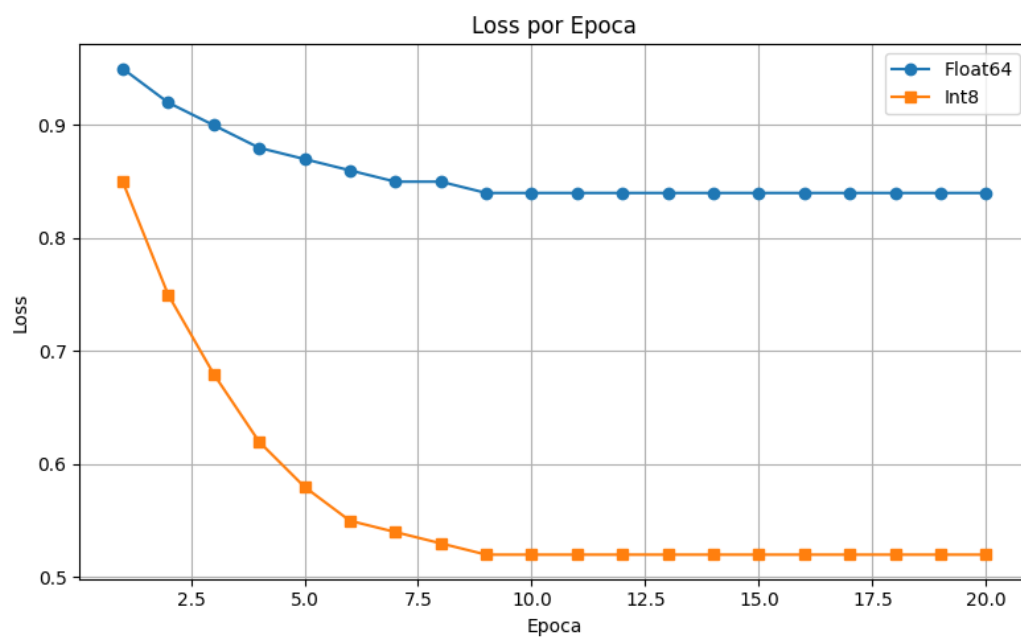
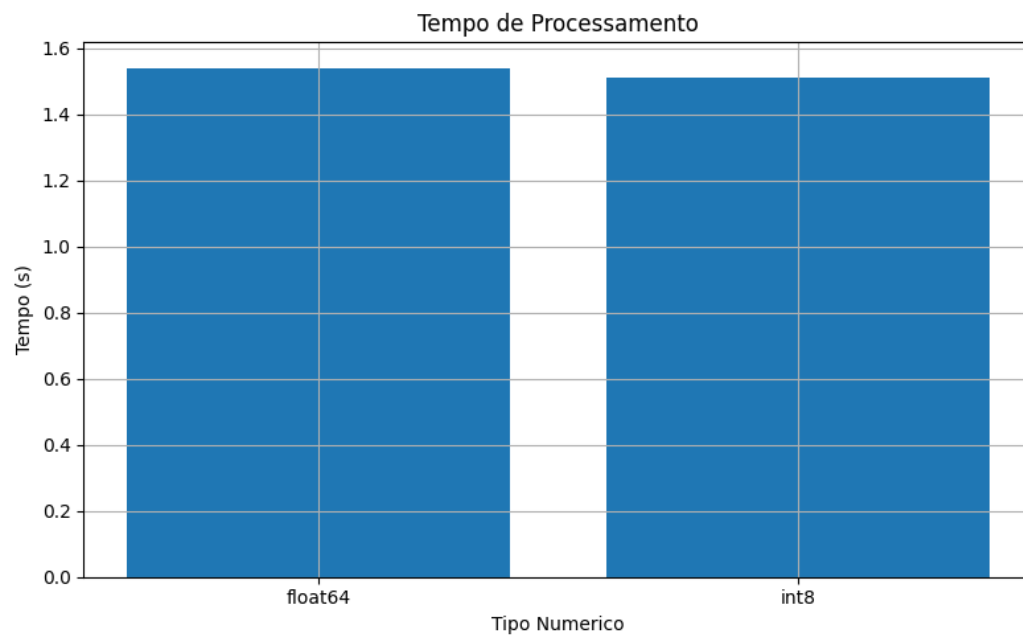


Grafico 3 - Comparacao de tempo de processamento



5. Influencia da Preemptao do Sistema Operacional

Sistemas operacionais preemptivos, como o Linux (utilizado neste experimento), utilizam escalonadores que interrompem processos em execucao para alocar o processador a outros processos considerados mais prioritarios. O escalonador CFS (Completely Fair Scheduler) do Linux, presente em Kernels 64 bits modernos, opera com base em fatias de tempo (time slices) e prioridades dinamicas.

Esta caracteristica pode influenciar os resultados de tempo observados nos treinamentos da seguinte forma:

- **Variacoes de medicao:** mesmo rodando o mesmo codigo repetidas vezes, o tempo total pode variar ligeiramente devido a interrupcoes do escalonador para outros processos do sistema (ex.: daemon de atualizacao, servicos de rede, antivirius).
- **Carga do sistema:** se outros processos competem por CPU durante o treinamento, o tempo medido pode ser maior do que o tempo efetivo de processamento da rede neural.
- **Precisao numerica versus overhead:** a conversao para int8, teoricamente mais leve em memoria, nem sempre resulta em menor tempo de execucao quando processada em CPU. Isso ocorre porque a CPU e otimizada para operacoes em ponto flutuante (float64), e o overhead de conversao para int8 pode, em alguns casos, anular o ganho de memoria - especialmente em ambientes com preemptao ativa, onde o tempo de resposta do sistema as interrupcoes se mistura com o tempo de execucao do calculo.

Em sistemas embarcados aeroespaciais reais, o Kernel e frequentemente configurado com preemptao reduzida ou opera como RTOS (Real-Time Operating System), justamente para garantir que tarefas criticas, como o calculo de trajetoria de pouso, nao sejam interrompidas por processos menos prioritarios. Isso resulta em tempos de inferencia mais estaveis e previsiveis - um fator essencial quando a resposta precisa ser apresentada em tempo real no mostrador do instrumento da cabine.

No presente experimento, ambos os padroes numericos (int8 e float64) apresentaram tempos de execucao muito proximos (1.51 s vs 1.54 s), o que e coerente com o fato de que, para a arquitetura x86_64 com Kernel Linux preemptivo, a CPU processa ambos os tipos de forma eficiente - sendo as diferencas de acuracia e loss mais influenciadas pela natureza dos dados (com ou sem ruido) do que pelo tipo numerico em si.

6. Documentacao

6.1 Repositorio GitHub

O codigo-fonte completo do projeto, incluindo o script main.py, o dataset, os graficos gerados e este relatorio, esta disponibilizado no repositorio GitHub:

[INSERIR LINK DO REPOSITORIO AQUI]

6.2 Arquivo PDF

Este relatorio esta disponivel em formato PDF no repositorio GitHub, na pasta relatorio/AgroShield_AI.pdf.

6.3 Estrutura do repositorio

global-solution-os/

- |— codigo/ (script main.py com a rede neural MLP)
- |— dataset/ (dataset agricola CSV com 200 amostras)
- |— graficos/ (accuracy.png, loss.png, benchmark.png)
- |— prints/ (screenshots da execucao)
- |— relatorio/ (AgroShield AI.docx e .pdf)
- |— README.md

Autores

Gabriel Borba - RM553187

Gabriel Souza Fiore - RM553710

Gustavo Gouvea Soares - RM553842

Pedro Henrique Mello Silva Alves - RM554223

Guilherme Santiago - RM552321